

Diagrama de Classes e Definições de Rotas: Belo Horizonte

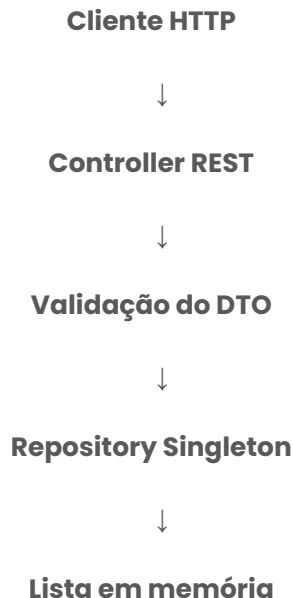
Autor: Esther Correia Pereira
Hiago Alecsander Santana Moreira
João Lucas Barros Botelho
Pedro Henrique Silva Freire
Pedro Henrique Campos Dourado
Thales Miguel Lima de Souza

Agenda

O objetivo desta apresentação é criar uma API REST para gerenciar peças e serviços da oficina Mecânica.

1. **Problema e escopo do MVP**
Organizar peças e serviços
2. **Arquitetura da solução**
Controller, DTO e repository
3. **Modelagem orientada a objetos**
Classes, enum e encapsulamento
4. **API REST e padrão Singleton**
Endpoints e armazenamento
5. **Testes e resultados**
26 testes, nenhuma falha

Escopo do MVP e arquitetura



- CRUD de peças e serviços;
- 10 endpoints REST;
- Códigos e datas gerados automaticamente;
- Validação dos dados de entrada;
- Respostas de erro padronizadas;
- Sem banco de dados, Spring Data ou @Autowired.

Modelagem orientada a objetos

- **Atributos privados e encapsulamento;**
- **Construtores;**
- **Getters e setters;**
- **Enumeração;**
- **Membros estáticos;**
- **Associação entre controller, repository e entidade.**

Peça

PecaController → PecaRepository «Singleton» → Peca →

CategoriaPeca

repository: PecaRepository

INSTANCE: PecaRepository

codigo: Long | quantidadeEstoque: Integer

categoria: CategoriaPeca

Serviço

ServicoController → ServicoRepository «Singleton» →

Servico

repository: ServicoRepository

INSTANCE: ServicoRepository

codigo: Long | nome: String

duracaoEstimadaMinutos: Integer

API REST, Singleton e demonstração



JAVA:

```
private static final PecaRepository INSTANCE = new PecaRepository();
```

```
private PecaRepository() {}
```

```
public static PecaRepository getInstance() {
```

```
    return INSTANCE;
```

```
}
```

Operação	Pecas	Serviços	Resposta de sucesso
Criar	POST /api/pecas	POST /api/servicos	201 Created
Listar	GET /api/pecas	GET /api/servicos	200 OK
Buscar	GET /api/pecas/{codigo}	GET /api/servicos/{codigo}	200 OK
Atualizar	PUT /api/pecas/{codigo}	PUT /api/servicos/{codigo}	200 OK
Excluir	DELETE /api/pecas/{codigo}	DELETE /api/servicos/{codigo}	204 No Content

Testes, resultados e limitações

- Indicadores principais:
 - 26 testes automatizados;
 - 0 falhas;
 - BUILD SUCCESSFUL;
 - testes de controller com MockMvc;
 - testes unitários dos repositories;
 - casos positivos, negativos e de borda.

Suíte	Testes
Contexto Spring Boot	1
Controller de peças	9
Controller de serviços	8
Repository de peças	4
Repository de serviços	4
Total	26

Conclusão e referências



- Conclusão:

- CRUD completo de peças e serviços;
- requisitos de POO e Singleton atendidos;
- API validada por testes e coleção Postman;
- documentação e UML disponíveis no repositório.

- Referências:

- Repositório GitHub do projeto;
- documentação oficial do Java;
- documentação oficial do Spring Boot;
- materiais da OAT I;
- collection Postman e documentação interna do projeto.